

Bloque 2 — Mini-especificaciones funcionales y técnicas

Banco y conciliación: B001 a B005

1. Propósito del bloque

Este bloque construye la infraestructura inicial del módulo bancario:

- Catálogo de bancos.
- Registro de cuentas bancarias.
- Configuración de formatos de cartola.
- Carga y validación de archivos CSV/XLSX.
- Creación de movimientos bancarios.
- Prevención de importaciones duplicadas.
- Prevención de movimientos duplicados.
- Registro de errores y resultados de importación.

Al finalizar este bloque, el sistema podrá recibir una cartola bancaria, interpretar sus columnas según el banco y convertirla en movimientos normalizados, todavía sin ejecutar la conciliación con facturas, rendiciones o factoring.

Las aplicaciones y conciliaciones de movimientos se desarrollarán en los bloques posteriores.

2. Resumen del bloque

Código	Actividad	Horas Gantt	Recurso	Cobertura actual	Resultado
B001	Catálogo de bancos	2 h	Backend	Cubierta	CRUD controlado de bancos
B002	Cuentas bancarias	4 h	Backend	Parcial	CRUD de cuentas y titular
B004	Plantillas de mapeo bancario	8 h	Integraciones	No cubierta	Configuración por banco y formato
B003	Importación de cartolas CSV/XLSX	10 h	Integraciones	Parcial	Validación, previsualización e importación
B005	Detección de duplicados	6 h	Backend	Parcial	Hash de archivo y fingerprint

El bloque suma **30 horas**.

La Carta Gantt define:

- B001 con el criterio "Bancos activos y datos consistentes".
- B002 con cuenta, moneda, banco, titular y estado.
- B003 con importación validada y resumen de errores.
- B004 con columnas configurables por banco y formato.
- B005 con hash de archivo y fingerprint por movimiento.

3. Ajuste recomendado a la secuencia de la Gantt

La secuencia actual aparece como:

B001 → B002 → B003 → B004 → B005

Sin embargo, una importación completa no puede realizarse correctamente antes de conocer:

- La fila que contiene los encabezados.
- La columna de fecha.
- La columna de cargo.
- La columna de abono.
- El formato de fecha.
- El separador decimal.
- La columna de descripción.
- La columna de referencia.
- Las filas que deben ignorarse.

Por eso, la secuencia funcional recomendada es:

B001 → B002 → B004 → B003 → B005

Predecesoras recomendadas

Tarea	Predecesora
B001	A007
B002	B001
B004	B002
B003	B004
B005	B003

Alternativamente, B003 podría dividirse conceptualmente en:

B003-A: Carga y lectura preliminar
B004: Configuración del mapeo
B003-B: Importación definitiva

Para mantener las 79 tareas existentes sin agregar actividades, es más simple mover B004 antes de B003.

4. Flujo funcional general

Registrar banco

- registrar cuenta bancaria
- definir plantilla del banco
- cargar cartola
- calcular hash del archivo
- leer encabezados
- aplicar plantilla
- normalizar filas
- validar datos
- detectar duplicados
- mostrar previsualización
- confirmar importación
- crear movimientos bancarios
- emitir resumen

La importación se realizará en dos fases:

Fase 1: validación

Archivo

- lectura
- transformación
- validación
- detección de duplicados
- resumen

En esta fase no se crean movimientos definitivos.

Fase 2: confirmación

Importación validada

- confirmación del usuario

- transacción atómica
- creación de movimientos
- actualización de contadores
- estado PROCESADA

Este diseño evita insertar parcialmente una cartola con errores desconocidos.

5. Cobertura actual del modelo

Los modelos existentes ya representan:

Banco
CuentaBancaria
ImportacionCartola
MovimientoBancario

Banco contiene nombre, código y estado activo. **CuentaBancaria** relaciona banco, número de cuenta, tipo, moneda, RUT del titular, estado y observaciones.

ImportacionCartola ya almacena:

- Cuenta bancaria.
- Rango de fechas.
- Archivo.
- Nombre del archivo.
- SHA-256.
- Total de movimientos.
- Estado.
- Observaciones.

También existe una restricción que impide repetir el mismo hash para una misma cuenta.

MovimientoBancario ya contiene fechas, tipo, monto, saldo, descripción, referencias, contraparte, fingerprint y estado de conciliación. Además, posee una restricción única por cuenta y fingerprint.

La base actual es adecuada, pero faltan:

1. Plantilla de mapeo.
2. Configuración de columnas.
3. Registro estructurado de errores.
4. Estado de importación validada.
5. Contadores detallados del proceso.
6. Servicio de lectura y transformación.
7. Previsualización antes de confirmar.

6. Modelos nuevos y ajustes requeridos

6.1 Plantilla de mapeo

Se propone incorporar:

```
class PlantillaMapeoCartola(ModeloAuditoria):
    class FormatoArchivo(models.TextChoices):
        CSV = "CSV", "CSV"
        XLSX = "XLSX", "Excel XLSX"

    banco = models.ForeignKey(
        Banco,
        on_delete=models.PROTECT,
        related_name="plantillas_cartola",
    )
    cuenta_bancaria = models.ForeignKey(
        CuentaBancaria,
        on_delete=models.PROTECT,
        null=True,
        blank=True,
        related_name="plantillas_cartola",
    )

    nombre = models.CharField(max_length=120)
    formato_archivo = models.CharField(
        max_length=10,
        choices=FormatoArchivo.choices,
    )

    nombre_hoja = models.CharField(max_length=100, blank=True)
    fila_encabezado = models.PositiveIntegerField(default=1)
    fila_inicio_datos = models.PositiveIntegerField(default=2)

    separador_csv = models.CharField(max_length=5, default=";")
    codificacion = models.CharField(max_length=30, default="utf-8-sig")

    formato_fecha = models.CharField(
        max_length=30,
        default="%d/%m/%Y",
    )
    separador_decimal = models.CharField(max_length=1, default=",")
    separador_miles = models.CharField(max_length=1, default=".")

    ignorar_filas_vacias = models.BooleanField(default=True)
    activa = models.BooleanField(default=True)
    version = models.PositiveIntegerField(default=1)
    observaciones = models.TextField(blank=True)
```

Alcance por banco o cuenta

Una plantilla podrá configurarse:

- Para todas las cuentas de un banco.
- Para una cuenta bancaria específica.

La plantilla de cuenta tendrá prioridad sobre la plantilla general del banco.

6.2 Campos de la plantilla

Se propone un segundo modelo:

```
class CampoMapeoCartola(ModeloAuditoria):
    class CampoDestino(models.TextChoices):
        FECHA_OPERACION = "FECHA_OPERACION", "Fecha operación"
        FECHA_CONTABLE = "FECHA_CONTABLE", "Fecha contable"
        FECHA_VALOR = "FECHA_VALOR", "Fecha valor"
        TIPO = "TIPO", "Tipo de movimiento"
        MONTO = "MONTO", "Monto único"
        MONTO_INGRESO = "MONTO_INGRESO", "Monto ingreso"
        MONTO_EGRESO = "MONTO_EGRESO", "Monto egreso"
        SALDO = "SALDO", "Saldo"
        DESCRIPCION = "DESCRIPCION", "Descripción"
        REFERENCIA = "REFERENCIA", "Referencia"
        NUMERO_DOCUMENTO = "NUMERO_DOCUMENTO", "Número documento"
        IDENTIFICADOR_EXTERNO = (
            "IDENTIFICADOR_EXTERNO",
            "Identificador externo",
        )
        CONTRAPARTE = "CONTRAPARTE", "Contraparte"
        RUT_CONTRAPARTE = "RUT_CONTRAPARTE", "RUT contraparte"
        CUENTA_CONTRAPARTE = (
            "CUENTA_CONTRAPARTE",
            "Cuenta contraparte",
        )

    plantilla = models.ForeignKey(
        PlantillaMapeoCartola,
        on_delete=models.CASCADE,
        related_name="campos",
    )
    campo_destino = models.CharField(
        max_length=40,
        choices=CampoDestino.choices,
    )
    columna_origen = models.CharField(max_length=150)
    obligatorio = models.BooleanField(default=False)
```

```
valor_defecto = models.CharField(max_length=255, blank=True)
orden = models.PositiveIntegerField(default=0)
```

Restricción

Una plantilla no podrá repetir el mismo campo de destino, salvo configuraciones expresamente admitidas.

6.3 Registro de errores

Se propone:

```
class ErrorImportacionCartola(ModeloAuditoria):
    class Nivel(models.TextChoices):
        ERROR = "ERROR", "Error"
        ADVERTENCIA = "ADVERTENCIA", "Advertencia"

    importacion = models.ForeignKey(
        ImportacionCartola,
        on_delete=models.CASCADE,
        related_name="errores",
    )
    fila = models.PositiveIntegerField(null=True, blank=True)
    columna = models.CharField(max_length=150, blank=True)
    codigo = models.CharField(max_length=50)
    nivel = models.CharField(
        max_length=20,
        choices=Nivel.choices,
        default=Nivel.ERROR,
    )
    valor_original = models.TextField(blank=True)
    mensaje = models.TextField()
    datos_fila = models.JSONField(default=dict, blank=True)
```

Ejemplos de códigos:

```
ARCHIVO_DUPLICADO
FORMATO_NO_SOPORTADO
HOJA_NO_EXISTE
COLUMNA_OBLIGATORIA_AUSENTE
FECHA_INVALIDA
MONTO_INVALIDO
TIPO_NO_RECONOCIDO
FILA_VACIA
MOVIMIENTO_DUPLICADO
MOVIMIENTO_POSIBLE_DUPLICADO
```

6.4 Ajustes a ImportacionCartola

Se recomienda agregar:

```
plantilla = models.ForeignKey(  
    PlantillaMapeoCartola,  
    on_delete=models.PROTECT,  
    null=True,  
    blank=True,  
    related_name="importaciones",  
)  
  
total_filas = models.PositiveIntegerField(default=0)  
total_validas = models.PositiveIntegerField(default=0)  
total_importadas = models.PositiveIntegerField(default=0)  
total_duplicadas = models.PositiveIntegerField(default=0)  
totalErrores = models.PositiveIntegerField(default=0)  
  
validada_en = models.DateTimeField(null=True, blank=True)  
procesada_en = models.DateTimeField(null=True, blank=True)
```

También se recomienda agregar un estado:

```
VALIDADA = "VALIDADA", "Validada"
```

El flujo será:

```
PENDIENTE  
→ VALIDADA  
→ PROCESADA
```

o:

```
PENDIENTE  
→ CON_ERRORES
```

Una importación procesada podrá pasar a **ANULADA** únicamente mediante el proceso controlado de reversa que se especificará en B016.

B001 — Catálogo de bancos

1. Objetivo

Permitir registrar y mantener las instituciones bancarias utilizadas por la empresa.

La tarea debe garantizar que los bancos activos tengan datos consistentes.

2. Modelo involucrado

Banco

Campos existentes:

Campo	Uso
nombre	Nombre visible de la institución
codigo	Código bancario o identificador interno
activo	Control de disponibilidad
Auditoría	Creación y modificación

El modelo ya impone unicidad para el nombre.

3. Ajustes recomendados

Normalización del nombre

Antes de guardar:

- Eliminar espacios iniciales y finales.
- Compactar espacios repetidos.
- Evitar duplicados por diferencias de mayúsculas.
- Mantener la forma visual oficial.

Ejemplo de registros que no deberían coexistir:

Banco Estado
BANCO ESTADO
Banco Estado

Código

El código:

- Será opcional.

- Se normalizará en mayúsculas.
- No podrá repetirse cuando esté informado.
- No deberá utilizarse como clave primaria.

Se recomienda una restricción condicional de unicidad para códigos no vacíos.

4. Casos de uso

- Listar bancos.
- Crear banco.
- Editar banco.
- Activar banco.
- Desactivar banco.
- Consultar cuentas asociadas.

No se permitirá eliminar físicamente un banco que tenga cuentas o movimientos asociados.

5. Reglas de negocio

1. El nombre es obligatorio.
2. No pueden existir nombres duplicados.
3. El código, cuando se informa, debe ser único.
4. Un banco inactivo no puede utilizarse para crear nuevas cuentas.
5. Un banco con cuentas activas no puede desactivarse directamente.
6. Para desactivar un banco deben desactivarse primero sus cuentas.
7. Los registros históricos seguirán mostrando el banco aunque esté inactivo.
8. No se utilizará borrado físico desde la interfaz.

6. URLs

```
/administracion/bancos/
/administracion/bancos/nuevo/
/administracion/bancos/<pk>/
/administracion/bancos/<pk>/editar/
/administracion/bancos/<pk>/activar/
/administracion/bancos/<pk>/desactivar/
```

7. Vistas

```
BancoListView
BancoDetailView
BancoCreateView
BancoUpdateView
BancoActivarView
BancoDesactivarView
```

Las acciones de activar y desactivar deberán realizarse mediante **POST**.

8. Formulario

```
class BancoForm(forms.ModelForm):  
    class Meta:  
        model = Banco  
        fields = [  
            "nombre",  
            "codigo",  
            "activo",  
        ]
```

El formulario deberá validar duplicados sin depender únicamente del error de la base de datos.

9. Template de listado

Columnas:

- Nombre.
- Código.
- Estado.
- Número de cuentas.
- Fecha de modificación.
- Acciones.

Filtros:

- Texto libre.
- Activos.
- Inactivos.

Acciones:

- Ver.
- Editar.
- Activar.
- Desactivar.

10. Permisos

```
view_banco  
add_banco  
change_banco  
desactivar_banco
```

Solo Administrador y Finanzas podrán modificar bancos.

11. Auditoría

Registrar:

- Creación.
- Cambio de nombre.
- Cambio de código.
- Activación.
- Desactivación.

La desactivación deberá aceptar una observación o motivo.

12. Pruebas

- Crear banco válido.
- Rechazar nombre vacío.
- Rechazar nombre duplicado con distinta capitalización.
- Rechazar código duplicado.
- Desactivar banco sin cuentas activas.
- Impedir desactivar banco con cuentas activas.
- Impedir eliminación con relaciones.
- Acceso con permiso.
- Acceso sin permiso.

13. Criterios de aceptación

- Los bancos se crean y editan correctamente.
- No existen duplicados funcionales.
- Los bancos inactivos no aparecen en formularios de nuevas cuentas.
- El historial conserva los bancos utilizados.
- No se elimina información asociada.
- Activaciones y desactivaciones quedan auditadas.

14. Evaluación de esfuerzo

Las **2 horas** de la Gantt son ajustadas.

Pueden ser suficientes para un CRUD básico usando vistas genéricas, pero no para implementar simultáneamente:

- permisos;
- auditoría;
- normalización;
- pruebas;
- interfaz completa.

Las funcionalidades transversales de A004, A005 y A007 reducen parte del trabajo requerido.

B002 — Cuentas bancarias

1. Objetivo

Registrar las cuentas bancarias controladas por la empresa y asociarlas con su banco, titular, moneda y estado.

2. Modelo involucrado

CuentaBancaria

Campos existentes:

- Banco.
- Nombre.
- Número de cuenta.
- Tipo.
- Moneda.
- RUT del titular.
- Estado.
- Observaciones.

Existe una restricción única por banco y número de cuenta.

3. Interpretación de campos

El campo actual:

nombre

deberá interpretarse como **alias de la cuenta**, por ejemplo:

Cuenta corriente principal
Cuenta pagos proveedores
Cuenta recaudación clientes

El campo no debe utilizarse como nombre del titular.

4. Campo faltante

La Gantt exige expresamente registrar el titular, pero el modelo actual solo contiene `rut_titular`.

Se recomienda agregar:

```
titular = models.CharField(  
    max_length=150,  
    blank=True,  
)
```

El resultado será:

Campo	Ejemplo
Alias	Cuenta corriente principal
Banco	Banco ejemplo
Número	123456789
Tipo	Cuenta corriente
Moneda	CLP
Titular	Extintores Real
RUT titular	12.345.678-9
Estado	Activa

5. Casos de uso

- Listar cuentas.
- Registrar cuenta.
- Editar cuenta.
- Ver detalle.
- Activar.
- Desactivar.
- Consultar importaciones.
- Consultar movimientos.
- Consultar conciliaciones.

6. Reglas de negocio

1. La cuenta debe pertenecer a un banco activo.
2. El número de cuenta es obligatorio.
3. El número debe normalizarse eliminando espacios y separadores visuales.
4. No puede repetirse el mismo número dentro del mismo banco.
5. La moneda será por defecto.
6. El RUT del titular debe validarse cuando esté informado.
7. El alias debe ser comprensible para el usuario.
8. Una cuenta inactiva no puede recibir nuevas cartolas.
9. Una cuenta no puede desactivarse si tiene:
10. Importaciones pendientes.

11. Conciliaciones abiertas.
12. Los movimientos históricos seguirán visibles.
13. No se permitirá eliminar físicamente una cuenta con información asociada.

7. Seguridad visual

En los listados se recomienda mostrar el número enmascarado:

.....6789

El número completo se mostrará únicamente:

- En el detalle.
- En edición.
- A usuarios autorizados.

8. URLs

```
/administracion/cuentas-bancarias/  
/administracion/cuentas-bancarias/nueva/  
/administracion/cuentas-bancarias/<pk>/  
/administracion/cuentas-bancarias/<pk>/editar/  
/administracion/cuentas-bancarias/<pk>/activar/  
/administracion/cuentas-bancarias/<pk>/desactivar/
```

9. Vistas

```
CuentaBancariaListView  
CuentaBancariaDetailView  
CuentaBancariaCreateView  
CuentaBancariaUpdateView  
CuentaBancariaActivarView  
CuentaBancariaDesactivarView
```

10. Formulario

Campos:

- Banco.
- Alias.
- Número de cuenta.
- Tipo.
- Moneda.
- Titular.
- RUT titular.
- Estado.

- Observaciones.

El selector de banco solo mostrará bancos activos.

11. Template de detalle

Secciones:

Identificación

- Banco.
- Cuenta.
- Alias.
- Tipo.
- Moneda.
- Titular.

Estado

- Activa o inactiva.
- Fecha de creación.
- Última modificación.

Resumen operacional

- Última cartola.
- Total de movimientos.
- Movimientos no conciliados.
- Conciliaciones abiertas.

Los indicadores podrán completarse en tareas posteriores.

12. Permisos

```
view_cuentabancaria  
add_cuentabancaria  
change_cuentabancaria  
desactivar_cuentabancaria  
ver_numero_cuenta_completo
```

13. Auditoría

Registrar:

- Creación.
- Cambio de número.
- Cambio de banco.
- Cambio de moneda.
- Cambio de titular.
- Activación.

- Desactivación.

Los cambios de banco o número deberán requerir confirmación, porque afectan la detección de duplicados.

14. Pruebas

- Crear cuenta en banco activo.
- Rechazar banco inactivo.
- Rechazar número duplicado.
- Normalizar número.
- Validar RUT.
- Desactivar cuenta sin procesos pendientes.
- Impedir desactivar con conciliación abierta.
- Impedir importación en cuenta inactiva.
- Enmascarar cuenta en listado.
- Ver número completo solo con permiso.

15. Criterios de aceptación

- Cada cuenta tiene banco, alias, número, moneda, titular y estado.
- No existen duplicados por banco.
- Las cuentas inactivas no aceptan importaciones.
- Los registros históricos se conservan.
- La información sensible se muestra de acuerdo con permisos.

B004 — Plantillas de mapeo bancario

1. Objetivo

Permitir que el sistema interprete cartolas con estructuras diferentes sin modificar el código Python para cada banco.

Esta tarea es esencial porque cada banco puede utilizar:

- Encabezados diferentes.
- Orden distinto de columnas.
- Fechas con formatos diferentes.
- Cargo y abono en columnas separadas.
- Un único monto con signo.
- Separadores decimales diferentes.
- Filas de resumen antes o después de los datos.
- Varias hojas dentro del archivo XLSX.

2. Modelos involucrados

Nuevos:

PlantillaMapeoCartola
CampoMapeoCartola

Relacionados:

Banco
CuentaBancaria
ImportacionCartola

3. Asistente de configuración

La creación de una plantilla se realizará mediante un asistente.

Paso 1 — Identificación

- Nombre de plantilla.
- Banco.
- Cuenta específica, opcional.
- Formato CSV o XLSX.
- Estado activo.

Paso 2 — Archivo de ejemplo

El usuario carga una cartola de muestra.

El archivo de ejemplo no crea movimientos.

Paso 3 — Estructura

Para CSV:

- Codificación.
- Separador.
- Fila de encabezado.
- Primera fila de datos.

Para XLSX:

- Nombre de hoja.
- Fila de encabezado.
- Primera fila de datos.

Paso 4 — Mapeo

Se presentan las columnas detectadas y los campos del sistema.

Ejemplo:

Columna de cartola	Campo del sistema
Fecha	Fecha operación
Detalle movimiento	Descripción
N.º operación	Identificador externo
Cargo	Monto egreso
Abono	Monto ingreso
Saldo	Saldo bancario

Paso 5 — Formatos

- Formato de fecha.
- Separador decimal.
- Separador de miles.
- Representación de ingresos.
- Representación de egresos.
- Textos que deben ignorarse.

Paso 6 — Previsualización

Se mostrarán entre 5 y 20 filas transformadas:

Fecha	Tipo	Monto	Descripción	Referencia
-------	------	-------	-------------	------------

Paso 7 — Guardar

La plantilla queda disponible para futuras importaciones.

4. Reglas de mapeo obligatorias

Toda plantilla debe definir:

- Fecha de operación.
- Descripción.
- Forma de obtener tipo y monto.

Debe utilizar uno de estos esquemas:

Esquema A — Monto y tipo separados

Columna tipo
Columna monto

Esquema B — Monto con signo

Monto positivo = ingreso
Monto negativo = egreso

Esquema C — Cargo y abono

Columna cargo = egreso
Columna abono = ingreso

No se permitirá guardar una plantilla sin un esquema monetario válido.

5. Versionado

Una plantilla utilizada por importaciones procesadas no debe editarse destruyendo su configuración histórica.

Al modificarla:

Versión 1 → inactiva
Versión 2 → activa

Cada importación debe conservar la plantilla y versión empleadas.

6. URLs

```
/administracion/plantillas-cartola/  
/administracion/plantillas-cartola/nueva/  
/administracion/plantillas-cartola/<pk>/  
/administracion/plantillas-cartola/<pk>/editar/  
/administracion/plantillas-cartola/<pk>/probar/  
/administracion/plantillas-cartola/<pk>/activar/  
/administracion/plantillas-cartola/<pk>/desactivar/
```

7. Vistas

```
PlantillaMapeoListView  
PlantillaMapeoCreateView  
PlantillaMapeoUpdateView  
PlantillaMapeoDetailView  
PlantillaMapeoProbarView  
PlantillaMapeoActivarView  
PlantillaMapeoDesactivarView
```

El asistente puede implementarse inicialmente mediante una sola vista con varias secciones o mediante sesiones temporales.

8. Servicios

```
services/importacion_cartolas/  
├─ plantillas.py  
├─ detectores.py  
├─ transformadores.py  
└─ validadores.py
```

Funciones principales:

```
detectar_columnas(archivo)  
validar_plantilla(plantilla)  
aplicar_plantilla(fila, plantilla)  
previsualizar_plantilla(archivo, plantilla)  
crear_nueva_version(plantilla)
```

9. Templates

```
templates/administracion/plantillas_cartola/  
├─ list.html  
├─ form.html  
├─ detalle.html  
├─ mapeo.html  
└─ previsualizacion.html
```

10. Permisos

```
view_plantillamapeocartola  
add_plantillamapeocartola  
change_plantillamapeocartola  
activar_plantillamapeocartola
```

11. Auditoría

Registrar:

- Creación.
- Cambio de configuración.
- Activación.
- Desactivación.
- Creación de versión.

- Prueba de una plantilla.
- Usuario que realizó la modificación.

12. Pruebas

- Plantilla CSV válida.
- Plantilla XLSX válida.
- Hoja inexistente.
- Columna obligatoria ausente.
- Formato de fecha incorrecto.
- Cargo y abono simultáneos.
- Fila sin monto.
- Separador decimal.
- Vista previa correcta.
- Nueva versión.
- Plantilla de cuenta con prioridad sobre la de banco.

13. Criterios de aceptación

- Las columnas se configuran sin modificar código.
- Se soportan CSV y XLSX.
- La plantilla puede probarse antes de activarse.
- La vista previa muestra datos normalizados.
- Las plantillas quedan versionadas.
- Cada importación conserva la versión utilizada.
- Una plantilla inválida no puede activarse.

B003 — Importación de cartolas CSV/XLSX

1. Objetivo

Permitir cargar una cartola, validarla con una plantilla y crear movimientos bancarios normalizados.

2. Modelos involucrados

```
ImportacionCartola
MovimientoBancario
PlantillaMapeoCartola
CampoMapeoCartola
ErrorImportacionCartola
```

El modelo actual ya conserva el archivo, SHA-256, cuenta, período, total de movimientos y estado.

3. Formatos admitidos

```
.CSV  
.xlsx
```

No se incluirá `.xls` en esta primera fase.

Para CSV se utilizará el módulo estándar de Python y para XLSX una biblioteca de lectura de hojas de cálculo como `openpyxl`.

Los archivos Excel deberán abrirse sin ejecutar macros ni fórmulas.

4. Pantalla de importación

Campos:

- Cuenta bancaria.
- Plantilla.
- Archivo.
- Fecha desde esperada.
- Fecha hasta esperada.
- Observaciones.

El selector de plantilla se filtrará según:

```
Cuenta elegida  
→ plantilla específica activa  
→ o plantilla general activa del banco
```

5. Proceso de validación

Paso 1 — Archivo

- Verificar extensión.
- Verificar tamaño.
- Calcular SHA-256.
- Comprobar archivo duplicado.
- Guardar `ImportacionCartola` como PENDIENTE.

Paso 2 — Lectura

- Abrir CSV o XLSX.
- Seleccionar hoja.
- Localizar encabezados.
- Verificar columnas.
- Recorrer las filas de datos.

Paso 3 — Normalización

Cada fila se transformará al formato interno:

```
{
  "fecha_operacion": date,
  "fecha_contable": date | None,
  "fecha_valor": date | None,
  "tipo": "INGRESO" | "EGRESO",
  "monto": Decimal,
  "saldo_bancario": Decimal | None,
  "descripcion_original": str,
  "referencia_bancaria": str,
  "numero_documento": str,
  "identificador_externo": str,
  "contraparte": str,
  "rut_contraparte": str,
  "cuenta_contraparte": str,
}
```

Paso 4 — Validación por fila

- Fecha válida.
- Tipo reconocido.
- Monto mayor que cero.
- Ingreso o egreso definido.
- No tener cargo y abono simultáneos.
- Rango de fecha consistente.
- Campos obligatorios presentes.
- Fingerprint calculable.

Paso 5 — Duplicados

- Dentro del mismo archivo.
- Contra movimientos existentes.
- Contra importaciones anteriores.

Paso 6 — Resultado

Se mostrará:

Indicador	Total
Filas leídas	N
Filas válidas	N
Duplicadas	N
Advertencias	N
Errores	N

6. Confirmación

Solo podrá confirmarse una importación cuando:

- No tenga errores bloqueantes.
- La cuenta esté activa.
- La plantilla esté activa.
- El archivo no haya sido procesado.
- El usuario tenga permiso.

La confirmación utilizará:

```
@transaction.atomic
def confirmar_importacion(importacion_id, usuario):
    ...
```

La importación deberá bloquearse con:

```
select_for_update()
```

para impedir dobles confirmaciones.

7. Política de errores

Errores bloqueantes

- Archivo duplicado.
- Formato no compatible.
- Plantilla inválida.
- Columna obligatoria ausente.
- Fecha imposible.
- Monto no interpretable.
- Tipo no reconocido.

Advertencias

- Campo opcional vacío.
- Descripción vacía.
- Referencia ausente.
- Movimiento ya existente.
- Fila de resumen ignorada.

En la primera versión, los errores bloqueantes impedirán confirmar toda la importación.

Los movimientos duplicados podrán omitirse, pero deberán aparecer claramente en el resumen.

8. Estados

```
PENDIENTE
  ↓ validar
VALIDADA
  ↓ confirmar
PROCESADA
```

Errores:

```
PENDIENTE
  ↓ error
CON_ERRORES
```

Anulación:

```
PENDIENTE o VALIDADA
  ↓ anular
ANULADA
```

La anulación de una importación ya procesada se tratará en B016.

9. URLs

```
/administracion/cartolas/
/administracion/cartolas/importar/
/administracion/cartolas/<pk>/
/administracion/cartolas/<pk>/validar/
/administracion/cartolas/<pk>/confirmar/
/administracion/cartolas/<pk>/errores/
/administracion/cartolas/<pk>/anular/
```

10. Vistas

```
ImportacionCartolaListView
ImportacionCartolaCreateView
ImportacionCartolaDetailView
ImportacionCartolaValidarView
ImportacionCartolaConfirmarView
ImportacionCartolaErroresView
ImportacionCartolaAnularView
```

11. Servicios

```
services/importacion_cartolas/  
├─ archivos.py  
├─ csv_reader.py  
├─ xlsx_reader.py  
├─ normalizacion.py  
├─ validacion.py  
├─ duplicados.py  
└─ importacion.py
```

Funciones:

```
calcular_sha256()  
leer_archivo()  
normalizar_fila()  
validar_fila()  
preparar_importacion()  
confirmar_importacion()  
generar_resumen()
```

12. Templates

```
templates/administracion/cartolas/  
├─ list.html  
├─ form.html  
├─ detalle.html  
├─ previsualizacion.html  
├─ errores.html  
└─ confirmar.html
```

13. Pantalla de previsualización

La tabla mostrará:

- Número de fila.
- Fecha.
- Tipo.
- Monto.
- Descripción.
- Referencia.
- Estado de validación.
- Mensaje.

No se mostrarán todas las filas en una sola página cuando el archivo sea grande.

14. Permisos

```
view_importacioncartola  
add_importacioncartola  
validar_importacioncartola  
confirmar_importacioncartola  
anular_importacioncartola  
descargar_archivo_cartola
```

15. Auditoría

Registrar:

- Carga.
- Validación.
- Confirmación.
- Anulación.
- Archivo.
- SHA-256.
- Cuenta.
- Plantilla utilizada.
- Número de movimientos creados.
- Número de duplicados.
- Número de errores.

16. Pruebas

Archivo

- CSV válido.
- XLSX válido.
- Extensión inválida.
- Archivo vacío.
- Archivo duplicado.
- Archivo demasiado grande.
- Hoja inexistente.

Datos

- Fecha válida e inválida.
- Monto con separador decimal.
- Cargo.
- Abono.
- Monto con signo.
- Fila vacía.
- Fila de totales.
- Descripción con caracteres especiales.
- Rango de fechas.

Proceso

- Validar sin crear movimientos.
- Confirmar importación válida.
- Impedir doble confirmación.
- Revertir transacción ante error.
- Registrar errores.
- Omitir duplicados controlados.
- Actualizar contadores.

17. Criterios de aceptación

- Se aceptan archivos CSV y XLSX.
- El archivo se valida antes de importar.
- Se presenta un resumen de errores.
- No se crean movimientos durante la previsualización.
- La confirmación es transaccional.
- Una importación no puede confirmarse dos veces.
- Los movimientos creados quedan relacionados con su importación.
- Se conservan archivo, plantilla, usuario, fecha y resultados.

B005 — Detección de duplicados

1. Objetivo

Evitar que la misma cartola o el mismo movimiento bancario sea cargado más de una vez.

El modelo ya incorpora dos controles de base:

1. Unicidad por cuenta y SHA-256 en `ImportacionCartola`.
2. Unicidad por cuenta y fingerprint en `MovimientoBancario`.

B005 debe completar estos controles con lógica de aplicación y mensajes comprensibles.

2. Nivel 1 — Duplicado de archivo

El SHA-256 se calculará usando los bytes originales del archivo.

```
cuenta bancaria + sha256
```

Si ya existe:

```
ARCHIVO_DUPLICADO
```

El sistema mostrará:

- Fecha de importación anterior.
- Usuario que la realizó.
- Estado.
- Cantidad de movimientos.
- Enlace al detalle.

No se permitirá confirmar nuevamente el archivo.

3. Nivel 2 — Duplicado exacto de movimiento

Opción preferente

Cuando el banco entrega un identificador único:

```
cuenta  
+ identificador_externo
```

Opción alternativa

Cuando no existe identificador bancario:

```
cuenta  
+ fecha_operacion  
+ tipo  
+ monto  
+ referencia  
+ numero_documento  
+ contraparte  
+ descripcion normalizada
```

El resultado se serializa de manera estable y se calcula:

```
SHA-256 → fingerprint
```

4. Normalización antes del fingerprint

- Fechas en formato ISO.
- Montos con dos decimales.
- Tipo en mayúsculas.
- Texto sin espacios extremos.
- Espacios repetidos compactados.
- Texto convertido a una forma consistente.
- Referencias vacías convertidas a cadena vacía.
- RUT normalizado.
- Cuentas sin separadores visuales.

Ejemplo conceptual:

```
base = "|".join([
    cuenta_id,
    fecha.isoformat(),
    tipo,
    f"{monto:.2f}",
    referencia_normalizada,
    documento_normalizado,
    contraparte_normalizada,
    descripcion_normalizada,
])
```

5. Nivel 3 — Duplicado dentro del mismo archivo

Antes de consultar la base de datos se mantendrá un conjunto temporal:

```
fingerprints_archivo = set()
```

Cuando se repita un fingerprint dentro del archivo:

```
MOVIMIENTO_DUPLICADO_EN_ARCHIVO
```

Esto evita que un mismo archivo genere una violación de unicidad al confirmarse.

6. Posibles duplicados

Puede existir el caso legítimo de dos movimientos con:

- Misma fecha.
- Mismo monto.
- Mismo tipo.
- Descripción idéntica.

Por eso se distinguirán:

Duplicado exacto

Coincide el fingerprint completo.

Resultado:

```
Se omite
```

Posible duplicado

Coinciden fecha, tipo, monto y contraparte, pero cambia alguna referencia.

Resultado:

Advertencia

En el MVP los posibles duplicados no se bloquearán automáticamente, salvo que la política del banco indique lo contrario.

7. Servicio

services/importacion_cartolas/duplicados.py

Funciones:

```
normalizar_texto()
normalizar_rut()
crear_fingerprint()
buscar_archivo_duplicado()
buscar_movimiento_duplicado()
detectar_duplicados_en_lote()
```

8. Consulta eficiente

No deberá ejecutarse una consulta por cada fila.

El flujo recomendado es:

```
Recolectar fingerprints del archivo
→ consultar fingerprints existentes con __in
→ construir conjunto de existentes
→ clasificar filas en memoria
```

Esto evita el problema de consultas N+1.

9. Seguridad concurrente

Aunque la validación previa indique que no hay duplicados, dos usuarios podrían confirmar archivos simultáneamente.

La restricción única de la base de datos será la última defensa.

El servicio deberá capturar:

IntegrityError

y convertirlo en un mensaje controlado, sin dejar una importación parcial.

10. Interfaz

La previsualización mostrará:

Fila	Movimiento	Resultado
12	Ingreso \$150.000	Válido
13	Ingreso \$150.000	Duplicado existente
14	Egreso \$45.000	Posible duplicado
15	Egreso \$20.000	Duplicado dentro del archivo

El resumen incluirá:

- Válidos.
- Duplicados existentes.
- Duplicados internos.
- Posibles duplicados.
- Errores.

11. Auditoría

Registrar:

- Archivo rechazado por duplicado.
- Movimiento omitido.
- Importación anterior relacionada.
- Fingerprint detectado.
- Usuario.
- Fecha.
- Decisión adoptada.

12. Pruebas

- Mismo archivo en la misma cuenta.
- Mismo archivo en cuenta diferente.
- Mismo identificador externo.
- Mismo movimiento en otra cartola.
- Duplicado dentro del archivo.
- Dos movimientos parecidos pero legítimos.
- Texto con espacios diferentes.
- Referencia con mayúsculas diferentes.
- Confirmaciones simultáneas.
- Rollback por `IntegrityError`.

13. Criterios de aceptación

- El mismo archivo no se procesa dos veces en una cuenta.
 - El mismo movimiento no se inserta dos veces.
 - Los duplicados se muestran antes de confirmar.
 - La detección no realiza una consulta por fila.
 - La base de datos conserva restricciones únicas.
 - Los conflictos concurrentes no crean datos parciales.
 - El usuario recibe un mensaje claro y puede consultar la importación original.
-

7. Estructura de archivos del bloque

```
administracion/
├── models/
│   └── bancos.py
├── forms/
│   ├── bancos.py
│   ├── cuentas_bancarias.py
│   ├── plantillas_cartola.py
│   └── importacion_cartola.py
├── views/
│   ├── bancos.py
│   ├── cuentas_bancarias.py
│   ├── plantillas_cartola.py
│   └── importacion_cartola.py
├── services/
│   └── importacion_cartolas/
│       ├── __init__.py
│       ├── archivos.py
│       ├── csv_reader.py
│       ├── xlsx_reader.py
│       ├── plantillas.py
│       ├── normalizacion.py
│       ├── validacion.py
│       ├── duplicados.py
│       └── importacion.py
├── selectors/
│   ├── bancos.py
│   ├── cuentas_bancarias.py
│   └── importaciones_cartola.py
├── templates/administracion/
│   └── bancos/
```

```
|   ├── cuentas_bancarias/
|   ├── plantillas_cartola/
|   └── cartolas/
|
└── tests/
    ├── test_bancos.py
    ├── test_cuentas_bancarias.py
    ├── test_plantillas_cartola.py
    ├── test_importacion_csv.py
    ├── test_importacion_xlsx.py
    └── test_duplicados_bancarios.py
```

8. Migraciones previstas

Migración 1 — Cuenta bancaria

- Agregar titular.
- Incorporar normalización o restricción de código de banco.

Migración 2 — Plantillas

- Crear PlantillaMapeoCartola.
- Crear CampoMapeoCartola.

Migración 3 — Importación

- Agregar plantilla.
- Agregar contadores.
- Agregar fechas de validación y procesamiento.
- Agregar estado VALIDADA.

Migración 4 — Errores

- Crear ErrorImportacionCartola.

Cada migración deberá probarse sobre una copia de la base existente.

9. Definición de terminado del bloque

B001-B005 se considerará terminado cuando:

- Se pueden registrar bancos.
- Se pueden registrar cuentas.
- Los bancos y cuentas pueden activarse y desactivarse.
- No se eliminan registros con historial.

- Existe una plantilla configurable.
- La plantilla admite CSV y XLSX.
- La plantilla puede probarse con un archivo de ejemplo.
- Se puede cargar una cartola.
- La cartola se valida sin crear movimientos.
- Los errores se muestran por fila.
- Se puede confirmar una importación válida.
- Los movimientos se crean dentro de una transacción.
- El archivo queda relacionado con sus movimientos.
- Se detectan archivos duplicados.
- Se detectan movimientos duplicados.
- No se generan consultas N+1.
- Se impiden confirmaciones dobles.
- Las operaciones quedan auditadas.
- Las pruebas automáticas del bloque pasan.

10. Riesgos técnicos del bloque

Riesgo	Impacto	Tratamiento
Formatos bancarios variables	Alto	Plantillas configurables
Fechas ambiguas	Alto	Formato definido por plantilla
Separadores monetarios	Alto	Normalización explícita
Cartolas duplicadas	Alto	SHA-256
Movimientos duplicados	Alto	Fingerprint y restricción
Archivos grandes	Medio	Límites y procesamiento por lotes
Fórmulas en Excel	Medio	Lectura sin ejecución
Importación parcial	Alto	Validación previa y transacción
Confirmación simultánea	Alto	Bloqueo y restricción única
Cambio posterior de plantilla	Alto	Versionado

11. Resultado consolidado

Al finalizar este bloque, el sistema contará con la siguiente cadena funcional:

```
Banco configurado
→ cuenta bancaria configurada
→ plantilla bancaria activa
→ archivo cargado
→ archivo validado
```

- errores identificados
- duplicados identificados
- importación confirmada
- movimientos bancarios disponibles

Estos movimientos serán la entrada para las siguientes tareas:

- B006 – Lista de movimientos
- B007 – Detalle de movimiento
- B008 – Clasificación manual
- B009 – Aplicaciones bancarias múltiples